EBU6304 Software Engineering Group Project

2025-26

Group number	100	
member 1	QM no: 231223689	Name: Yifan Tang
member 2	QM no: 231223690	Name: Junjie Chen
member 3	QM no: 231223678	Name: Wenjun Du
member 4	QM no: 231223667	Name: Bingjun Han
member 5	QM no: 231223645	Name: Mingxuan Cai
member 6	QM no: 231223601	Name: Yiben Liu

Final short report

1. Design

The design strategy for **TA&MO** was **requirements-driven, role-centred and workflow-oriented**. Instead of beginning with interface screens or implementation technologies, we first considered the recruitment process as a set of user goals, responsibilities and quality constraints. The system needed to support three clearly different stakeholder groups — applicants/TAs, module organisers and administrators — so the design strategy focused on separating their responsibilities while still connecting them through one coherent recruitment workflow. We therefore followed a top-down strategy: identify externally visible requirements, map them to role-specific user journeys, define clear module boundaries, and then design the architecture, data model and interfaces around those boundaries. Security, maintainability, usability and reliability were treated as guiding design concerns from the beginning, rather than as later implementation details.

At architecture level, the system uses a pragmatic **Model–View–Controller style** suitable for a Java Servlet/JSP project. JSP pages and vanilla JavaScript form the view layer; role-specific servlets act as controllers exposing REST-like endpoints; service classes contain business rules; model classes represent persistent domain entities; and `JsonDataService` provides a single persistence gateway to `data.json`. This separation of concerns improved maintainability because UI changes, workflow rules, and storage access could usually be changed independently.

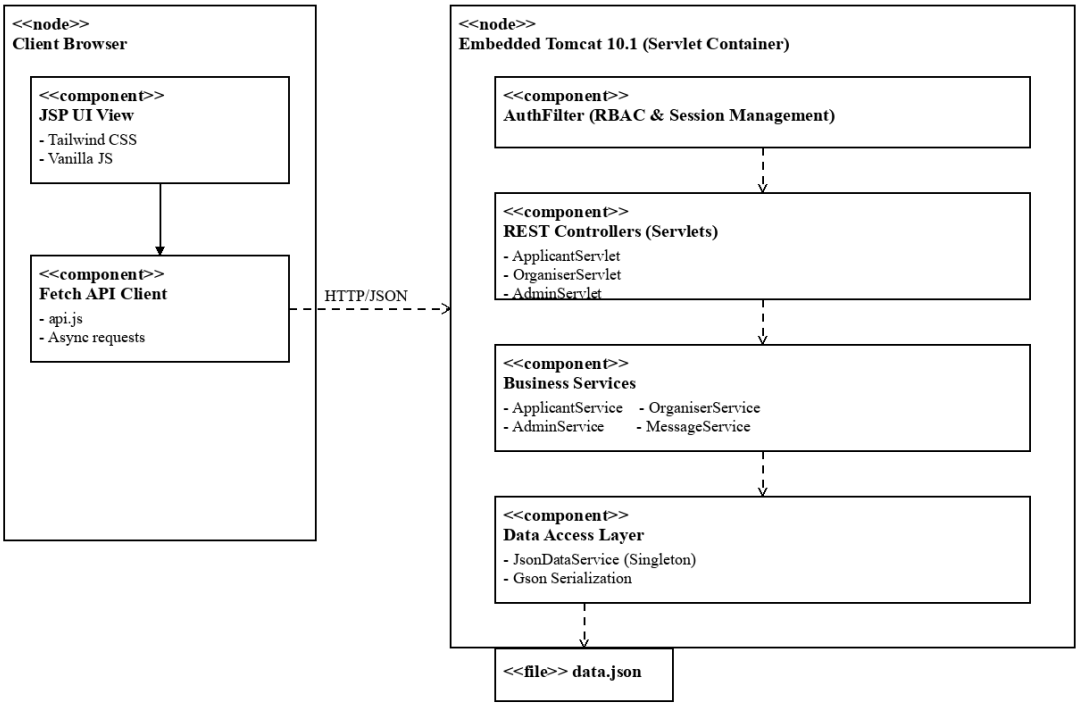


Figure 1. System architecture showing layered responsibilities (UML Component Diagram).

The component design deliberately uses **high cohesion and low coupling**. `ApplicantService` owns applicant concerns such as job browsing, profile editing, document management and application submission. `OrganiserService` owns job posting, recruitment pipeline, interview scheduling and applicant review. `AdminService` owns global management, data editing, workload monitoring and audit-style operations. `MessageService` encapsulates conversation logic, including unique contact lists and user detail cards. This structure avoids a “god servlet” and makes each component easier to reason about, test and extend.

The data model captures the recruitment lifecycle as a set of related entities. A `Job` belongs to a module and organiser and includes explicit `workDates`; an `Application` links an applicant to a job and progresses through applied, review, shortlisted, interview, offer and rejected states; a `Profile` stores academic details, skills, coursework, experience and reusable documents; `Interview` and `Notification` provide scheduling and status feedback; and `Message` supports communication between applicants and organisers. The model therefore represents both static information and process state.

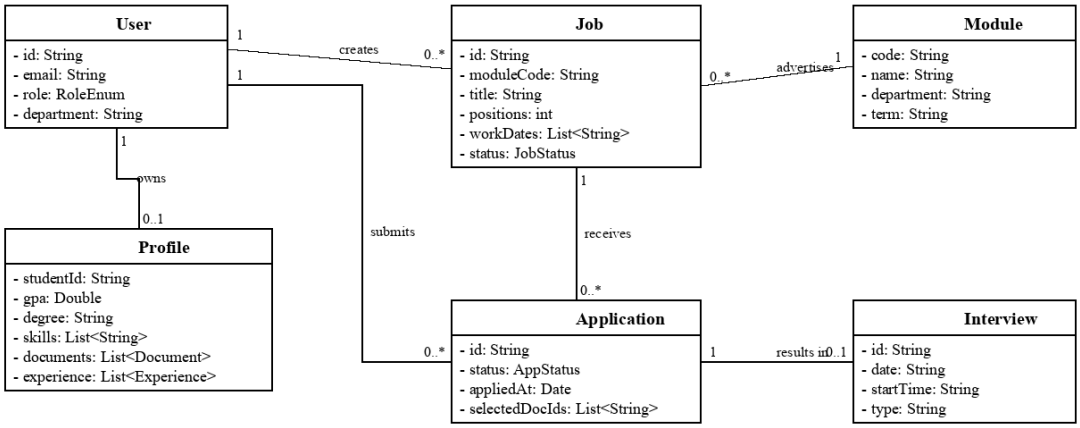


Figure 2. Core domain model supporting the TA recruitment lifecycle (UML Class Diagram).

Several design principles were applied. **Role-Based Access Control (RBAC)** is enforced through `AuthFilter`, which protects `/api/applicant/*`, `/api/organiser/*` and `/api/admin/*` according to session role. **Defensive validation** is placed server-side for permissions, duplicate applications, invalid status transitions and ownership checks, while the front end adds immediate usability feedback. **Information hiding** is used in profile retrieval and messaging: sensitive fields such as passwords are not exposed to the UI. **Single source of truth** is maintained by routing all persistence through `JsonDataService`, reducing inconsistent file access.

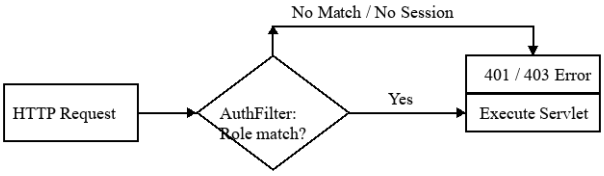


Figure 3. Role-Based Access Control logic enforcing permission boundaries (Flowchart).

The UI design supports user workflows instead of isolated pages. Applicants can browse jobs, view work schedules, apply with pre-filled profile data, manage CV documents, track applications and receive offers. Organisers can create jobs with generated work dates, preview submitted documents, inspect full applicant profiles, manage a Kanban-style pipeline and schedule interviews. Administrators can maintain users, modules, departments, terms, workload and raw data. These workflows demonstrate usability and traceability from stakeholder needs to implemented features.

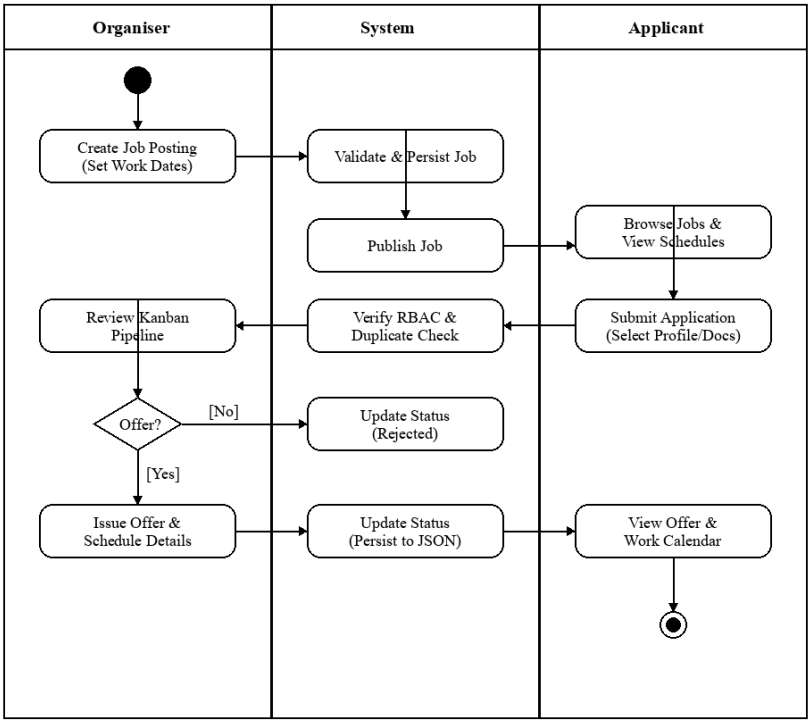


Figure 4. End-to-end recruitment workflow mapping user actions to system boundaries (UML Activity Diagram).

Quality attributes were considered using ISO/IEC 25010-style thinking. **Functional suitability** is addressed by covering the full recruitment workflow. **Security** is addressed by session authentication, RBAC and ownership checks. **Maintainability** is supported by layered services and consistent API patterns. **Usability** is supported by role-aware navigation, inline validation, modals, progress indicators and meaningful error messages. **Reliability** is improved through duplicate prevention, cascade warnings in admin deletion, and consistent JSON persistence.

2. Implementation

The implementation strategy followed an **incremental and iterative** approach. We first delivered a working vertical slice: login, role-aware routing, basic dashboards and JSON persistence. We then expanded the system by role: applicant profile and applications, organiser job management and recruitment pipeline, administrator data management and workload monitoring, and finally cross-cutting features such as notifications, messaging, document upload/preview and work schedules. This reduced integration risk because every iteration produced executable software rather than a large untested final merge.

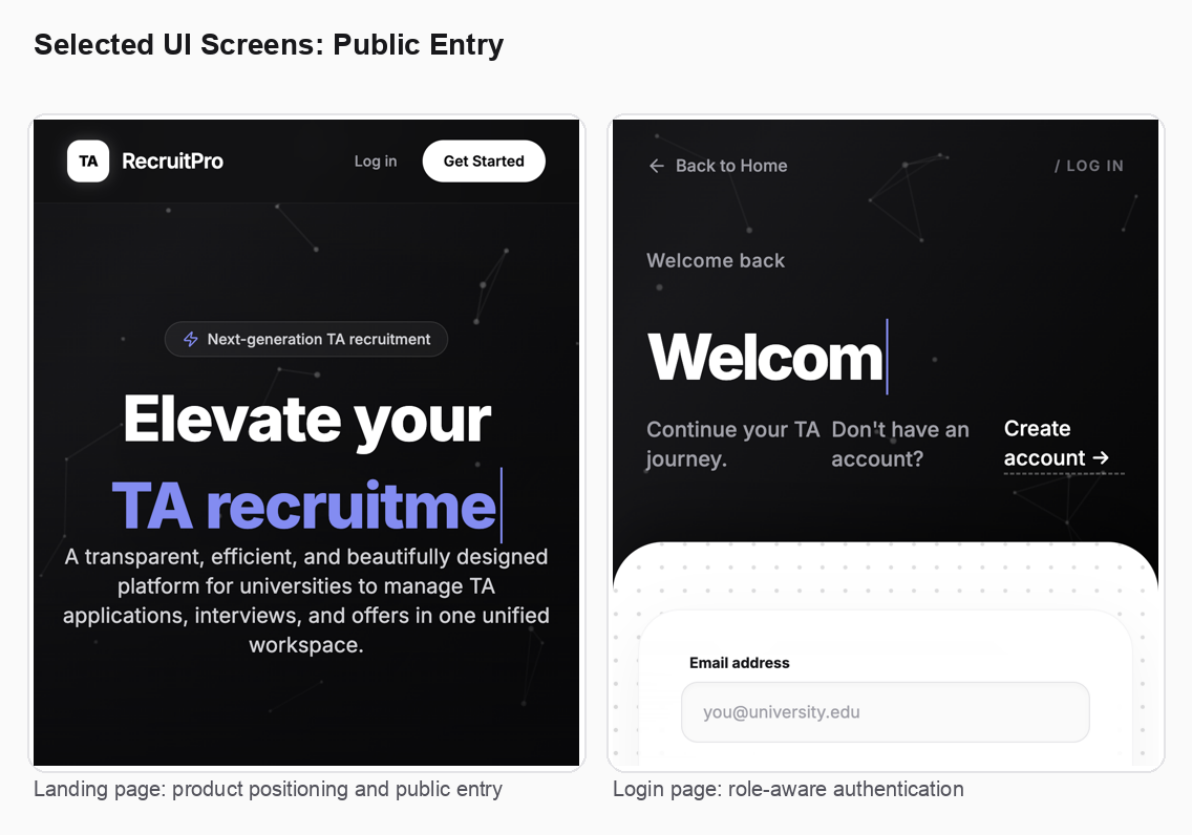


Figure 5. Public entry and login screens demonstrating polished UI and role-aware authentication.

The project is built with **Java 17, Maven, Embedded Tomcat 10.1, Jakarta Servlet/JSP, Gson, Tailwind CSS, Lucide icons and vanilla JavaScript**. We chose this stack because it matches the course requirement for web software engineering while remaining transparent enough to show architecture, request handling and persistence clearly. Maven provides a repeatable build process (`mvn compile exec:java`), while embedded Tomcat simplifies deployment by running the application locally under `/tamo`.

Implementation was organised around role-specific APIs. Applicant endpoints support dashboard statistics, job search/filtering, job details, application submission, application history, profile update and document management. Organiser endpoints support managed jobs, pipeline status movement, application details, interview scheduling and notification handling. Admin endpoints support global analytics, user/module/term/department management, workload reassignment, logs and a data editor. This API organisation makes the system easier to understand because endpoint location mirrors stakeholder responsibility.

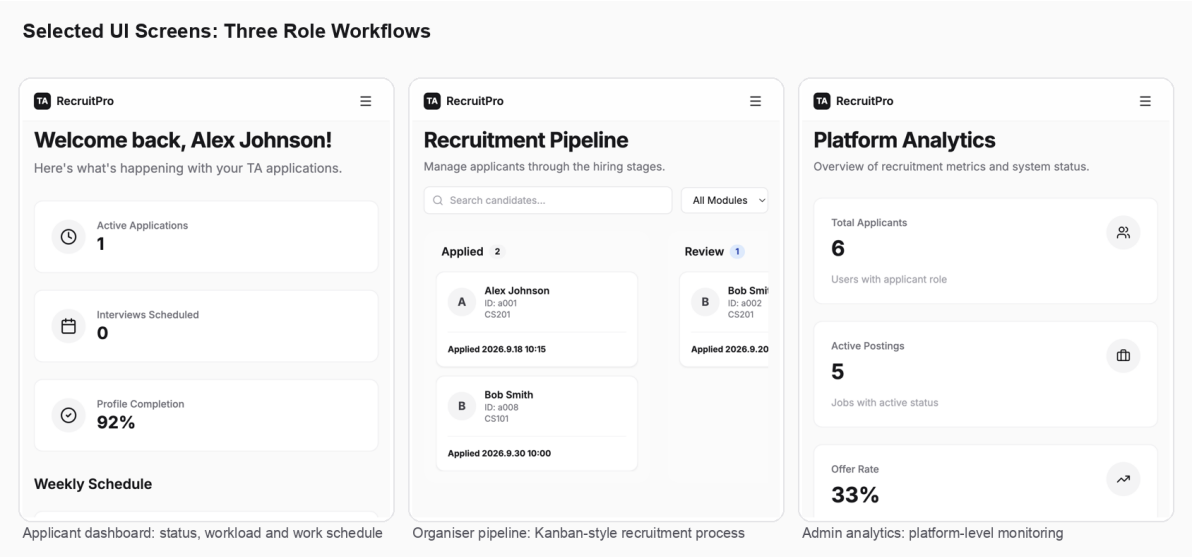


Figure 6. Representative role-specific interfaces for applicant, organiser and administrator workflows.

Important implementation details include: session-based authentication in `AuthFilter`; safe JSON serialization with Gson; server-side permission checks before document deletion and application review; dynamic front-end calls through `static/js/api.js`; reusable JSP layout fragments; and browser-side enhancement for document preview. For example, document preview uses native PDF/image preview and `mammoth.js` for `.docx` conversion, while `.doc` files fall back to a clear download recommendation because legacy binary Word format cannot be reliably rendered in the browser.

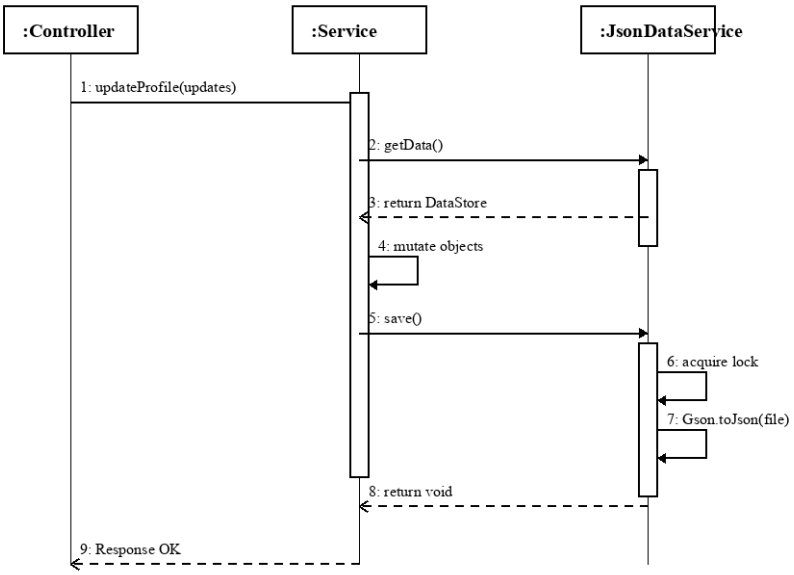


Figure 7. Data persistence mechanism ensuring single source of truth via JsonDataService (UML Sequence Diagram).

The work schedule feature illustrates implementation from requirement to data model. The organiser UI allows quick generation of dates such as "12 weeks every Wednesday"; generated dates are stored in `Job.workDates`; applicants see the schedule in job details; and accepted applicants see work dates on the dashboard. A timezone defect was found and fixed by replacing `toISOString()` formatting with local date construction, demonstrating the importance of real-world edge cases even in simple-looking date logic.

Version control was central to collaboration throughout the project. The team used a **structured Git branching strategy**, including a shared dev branch and role-based feature branches such as feature-admin, feature-MO and feature-applicant. Each branch was used to develop, modify or remove functionality related to a specific part of the system, so that team members could work independently on administrator, module organiser and applicant features without interfering directly with each other's code. Bug fixes and functional improvements were also committed to the relevant feature branches before being reviewed and merged back into the shared development branch.

We used pull and fetch regularly to keep local work aligned with the latest shared version, and we resolved integration issues through selective merging when a feature branch contained unrelated files or overlapping changes. This branching strategy improved collaboration efficiency because it kept code changes organised by responsibility, reduced the risk of conflicts, and made communication between team members clearer. Runtime data files and uploaded documents were treated carefully so that demo data and local test artifacts did not accidentally overwrite code changes. Commit history also provided useful traceability from defect reports to fixes, such as profile editing, notification 403 errors, document preview and schedule generation.

The project build plan can be summarised as: establish base architecture and authentication; implement role dashboards; implement core recruitment flows; add admin management and audit support; add communication, document and notification features; then stabilise with bug fixing and demo data. This plan

reflects agile values: frequent integration, feedback-driven change, and continuous refinement of user-facing workflows.

3. Testing

Our testing strategy was designed to **verify both the functional correctness and the quality of the TA&MO recruitment workflow**. Instead of only checking individual pages, we tested the system from the perspective of requirements, user roles, system integration and previously fixed defects. This strategy was chosen because TA&MO has three distinct stakeholder groups — applicants, module organisers and administrators — and each role interacts with different parts of the recruitment process. Therefore, the testing needed to confirm not only that each feature worked independently, but also that the complete workflow remained consistent across roles, APIs, services and JSON persistence.

The main testing methods included **requirements-based testing, role-based end-to-end testing, integration testing through API calls, UI interaction testing, positive and negative test cases, and regression testing after bug fixes.** Requirements-based testing checked whether each major requirement had been implemented and could be observed in the running system, such as job application, profile editing, pipeline management and admin data maintenance. Role-based testing verified that applicants, organisers and administrators could only access and perform actions appropriate to their permissions. End-to-end testing followed complete user journeys, for example an applicant applying for a job, an organiser reviewing the application, scheduling an interview and sending a status update. Integration testing checked that JSP pages, JavaScript API calls, servlets, service classes and JsonDataService worked together correctly. Negative and regression testing were used to confirm that invalid actions, previous bugs and edge cases were handled safely.

Test case design used both **positive and negative paths**. **Positive tests** confirmed normal workflows: applicant login, profile update, CV upload, job application, organiser job creation, pipeline movement, interview scheduling, admin user/module management and messaging. **Negative tests** covered invalid login, missing required job fields, duplicate application, unauthorised role access, invalid document deletion, backward movement in the pipeline, unsupported document preview and invalid data editor operations. This gives stronger evidence than only demonstrating happy paths.

Specific example 1: RBAC and notification bug. A 403 error occurred because a shared layout called `applicantApi.getNotifications()` even when an organiser was logged in. The fix was to select the notification API dynamically according to the session role. The regression test was to log in as applicant and organiser separately, load pages using the shared header, and confirm each role calls its own notification endpoint without a 403.

Specific example 2: document ownership bug. Applicants could not delete their uploaded CV because document IDs were not globally unique enough for the original search logic. The corrected implementation first locates the current user's profile and then searches documents inside that profile. The test was to upload a document, delete it as the same applicant and confirm success, while ensuring another applicant cannot delete it.

Specific example 3: work schedule date generation. When an organiser selected Wednesday, generated dates appeared as Tuesday because `toISOString()` converted local midnight into UTC. The fix used local date parsing and manual `YYYY-MM-DD` formatting. The test cases included generating 12 weeks from a known Tuesday, Wednesday and Sunday start date and checking that the label and date remain consistent across the browser timezone.

Specific example 4: application detail completeness. Organisers originally saw limited applicant data. We extended `getApplicationDetails` to return email, phone, department, student ID, year, max GPA, skills and experience, then tested that the applicant profile card renders those fields and still hides sensitive fields such as password. This validates both functional completeness and information security.

Specific example 5: requirements-based job application testing. One core requirement was that an applicant should be able to browse available TA jobs, view job details and submit an application using their profile information. The test was to log in as an applicant, open the job list, filter or select a job, review the job details and submit an application. The expected result was that the application was created successfully, linked to the correct applicant and job, and displayed in the applicant's application history. This validates that the implemented feature directly satisfies a visible user requirement.

Specific example 6: data persistence testing. Since the system uses JSON-based persistence, we tested whether changes remained available after page reloads and across different user roles. For example, an organiser created a new job with work dates, then the page was refreshed and the applicant job list was opened. The expected result was that the new job still existed, appeared to applicants, and retained the correct module, organiser and work schedule data. This validates that front-end actions, service-layer logic and JsonDataService persistence work together correctly.

Testing results show that the major workflows are operational in the demo environment. The final demo data supports applicants, organisers and admin accounts; role dashboards load correctly; job search/filtering works; profile editing persists; applications move through the pipeline; interviews can be scheduled; notifications load according to role; and the admin data editor exposes new fields such as `workDates`, profile documents and experience. Remaining risks are mainly those expected in a course prototype: the JSON-file store is not as robust as a database under high concurrency, browser rendering of legacy `.doc` files is limited, and future work should add automated unit tests and CI checks for services such as status transitions and data validation.

4. The use of Generative AI (GenAI)

We used Generative AI as an **engineering assistant**, not as an unchecked replacement for design judgement. It supported several stages of the software development process: requirement clarification, UI copy refinement, code generation for repetitive JSP/JavaScript structures, debugging, test idea generation, commit-message drafting and report writing. This aligns with a modern software engineering workflow where AI can accelerate iteration while developers remain responsible for correctness, maintainability and ethical use.

During requirements analysis, GenAI helped translate informal stakeholder needs into concrete user stories and acceptance criteria. For example, the vague requirement "MO should know who they are chatting with" became a feature in which clicking a chat avatar opens a role-aware user information card. The requirement "TA should see what information is sent when applying" became a pre-filled application form showing profile details and selected documents before submission. This helped improve requirement testability, a key software engineering quality.

During design and implementation, GenAI was useful for proposing code-level changes across related files. For example, adding work schedules required coordinated updates to the `Job` model, organiser service, job creation UI, applicant job detail page and dashboard display. AI assistance reduced the mechanical cost of tracing these dependencies. It also helped identify common web pitfalls, such as using local date formatting instead of UTC `toISOString()` for schedule generation.

During debugging, GenAI was most effective when combined with runtime evidence. Error messages such as JSP compilation failures, HTTP 403 responses and Tomcat logs were used as evidence, and fixes were applied only after checking the relevant source files. This prevented blind changes. Examples include fixing `layout-bottom.jsp` after discovering that JSP include scope did not expose `userRole`, and fixing organiser/applicant notification calls based on the actual endpoint and RBAC behaviour.

The limitations were also clear. GenAI can hallucinate APIs, assume unavailable libraries, or suggest a fix that works in theory but not in the local environment. For instance, Office Online Viewer was not suitable for previewing local `localhost` documents, so the design was revised to use browser-side `.docx` conversion

with `mammoth.js` and a clear fallback for `.doc`. AI outputs also required careful review to avoid over-engineering, preserve existing project style and prevent unrelated changes from entering commits.

Our critical evaluation is that GenAI was valuable for **speed, breadth and explanation**, but its effectiveness depended on disciplined software engineering practices: reading the existing code, making small changes, compiling, testing manually, checking Git status, and preserving separation of concerns. Used in this way, GenAI improved productivity and helped us articulate design rationale, but final accountability remained with the team.

Individual contribution and reflection

Member 1 – Backend architecture and persistence

QM no: 231223689

Name: Yifan Tang

Main contribution: I contributed mainly to the backend architecture, data model and persistence layer. I helped design the role-based API structure for applicants, organisers and administrators, implemented service-layer logic for profiles, job applications, recruitment pipeline operations and JSON persistence, and fixed defects related to document ownership, notification permissions and work schedule date generation. I also supported Git integration by keeping the `dev` branch updated and resolving conflicts safely.

Reflective statement: This project helped me understand that software engineering is not only about writing features, but about designing changeable systems. The biggest challenge was coordinating many related components when one requirement changed, such as adding work schedules across the model, service and UI. I improved my skills in debugging with logs, tracing data flow, managing Git branches, and thinking about maintainability, security and testability before coding.

Member 2 – Applicant experience and profile workflow

QM no: 231223690

Name: Junjie Chen

Main contribution: I worked on administration features such as global user/module management, data editor schemas, workload monitoring, logs, terms and department configuration. I also helped improve the data editor so newly added fields such as `workDates`, profile documents and experience remain visible and editable for admin users.

Reflective statement: I gained a deeper appreciation of maintainability and operational support. Admin tools are easy to overlook, but they are essential for diagnosing issues and maintaining data quality. I learned how cascade impacts, audit logs and validation rules help protect a system after deployment, not only during development.

Member 3 – Organiser workflow and recruitment pipeline

QM no: 231223678

Name: Wenjun Du

Main contribution: I was responsible for the UI design and functional implementation of the MO (Module Organiser) interface. This included displaying basic statistics, interview information, and module details on the dashboard, as well as module creation, updating applicant statuses, and viewing applicants' detailed profiles. I also implemented resume preview and download functionality, along with core features such as creating, editing, and deleting interviews, and deleting modules.

Reflective statement: This project helped me understand the rigorous workflow of software development from design to implementation, as well as various design details. I learned how different interfaces are interconnected and how they interact with each other. It enhanced my teamwork and communication skills, and also developed my ability to focus on details.

Member 4 – Login and registration interface features

QM no: 231223667

Name: Bingjun Han

Main contribution: I worked on the homepage, registration and login features of the system. For the homepage, I helped build the landing page structure, navigation, visual sections and interactive front-end effects, including particle background, typewriter text, stitch matrix animation and card tilt effects. For registration and login, I implemented the user interface, role selection, form validation and front-end API calls through api.js. I also connected the pages with the backend authentication flow, where user information is sent to AuthServlet, processed by AuthService, and stored or verified through data.json. In addition, I helped ensure successful users are redirected to the correct dashboard based on their role.

Reflective statement: I worked on the homepage, registration and login features of the system. For the homepage, I helped build the landing page structure, navigation, visual sections and interactive front-end effects, including particle background, typewriter text, stitch matrix animation and card tilt effects. For registration and login, I implemented the user interface, role selection, form validation and front-end API calls through api.js. I also connected the pages with the backend authentication flow, where user information is sent to AuthServlet, processed by AuthService, and stored or verified through data.json. In addition, I helped ensure successful users are redirected to the correct dashboard based on their role.

Member 5 – Lead Developer, Teaching Assistant Recruitment Module

QM no: 231223645

Name: Mingxuan Cai

Main contribution: I mainly worked on the TA-related features, including job browsing, position details, application submission, application tracking, applicant profiles, document uploads, and schedule/workload information. I helped connect applicant data with organiser review pages so skills, experience, CVs, transcripts, and application status could be viewed clearly during recruitment.

Reflective statement: I learned that a TA recruitment system needs to be clear and reliable for both applicants and organisers. Features such as profile completion, application stages, and schedule previews help users make better decisions and reduce confusion. This work improved my understanding of user-centred design and smooth data flow between different roles.

Member 6 –Optimized TA Recruitment: Better Role Coordination & Clearer Info Flow

QM no: 231223601

Name: Yiben Liu

Main contribution: I mainly worked on TA-related features, including applicant pages, application management, profile information, workload visibility, schedule information, and links with organiser review functions. I improved how TA data is displayed and updated across the system, helping applicants track their progress and organisers review applicant details more clearly.

Reflective statement: I learned that a TA recruitment system needs clear information flow between applicants, organisers, and status changes. This improved my understanding of role-based design, user-centred display, and consistent data across modules.